

Learning Location, Not Looking: Auditing Placement Memorization in a 30M-Parameter Language-Conditioned Policy

Anonymous submission

Abstract

Fixed-placement benchmarks let imitation policies learn *location* instead of *looking*. On LIBERO-Object we measure the preconditions exactly, from the shipped init files: in 6 of 10 tasks the target’s start position is constant to float64 rounding (two values, 1 ULP apart, $\sim 3 \times 10^{-17}$ m), its *orientation* is bit-identical across all 50 states in **all ten**, and the ten tasks between them use just **two** table positions, 22 cm apart. A policy that memorizes two coordinates can present as generalizing over ten. Measuring all four suites locates the defect rather than generalizing it: placement entropy is 2.11 bits/task in LIBERO-Object against 5.50–5.62 at a 5.644-bit ceiling in Spatial, Goal and Long—the frozen suite is precisely the object-identity suite, where a coordinate memorizer is both admissible and maximally misleading. Orientation, by contrast, is bit-identical in 37 of 38 resolvable tasks across all four.

We audit our own stack and find placement memorization at **four** layers: a calibrated expert whose offset constant encodes pose, an iteration-coupled selection loop, a grasp head whose regression prefers proprioception to pixels, and a place head that memorized a command→location map against a basket that never moves. Three are addressed by *data* rather than architecture: ten placement-teleported episodes, command coverage in training, and DAgger on the machine’s own deployment viewpoints — though command coverage is a repair we partly walk back here: it demonstrably fixes the place *point* (14.15 cm→0.78 cm spread) while its effect on swap *behaviour* did not replicate when we powered the cell up. The released head scores 0.700 held-out (35/50 [0.56, 0.81]) and 0.520 randomized (26/50) where *our* free per-tick regression baseline, under matched trunk and corpus, scores 0/56—a statement about that baseline, not about the policy class. Anchoring that comparison: the same shell driven by *random* goals scores 0/10 and by *oracle* goals 10/10, so the shell neither solves the task alone nor caps it. A third object crosses at 22/50 [0.31, 0.58] against a same-seed control at 3/50 (exact McNemar $p < 10^{-4}$). Twin control cells scope that verification from both sides: the behavioural certificate is joint in (*head, stack*)—a detector-stack rebuild inverts which head looks better. A second, faithful rebuild with every package pinned reproduces the reference band (8/10 vs a recorded 7/10), but exposes a component no requirements file pins: holding seed, trial and weights fixed and changing only MuJoCo’s GL backend moves the episode (246 vs 250 steps), so MUJOCO_GL belongs in any reported stack.

The platform is 30.2 M parameters deployed with *no* language model—an open-vocabulary detector’s own text tower supplies every text embedding. Size is the setting that makes an end-to-end trace feasible, not a claim; we discuss where it sits in the compact-VLA range in §6.2, and nothing below is attributed to it.

That VLAs ignore language is by now well established, and our system confirming it is not the contribution. What a system this small permits is saying *where the language goes*. An annotation-free probe comparing two instructions’ detections on one frame localizes the loss to the grounding stage; a 2×2 decomposition driving detection prompts and place-head embedding from different instructions isolates it to one channel with no interaction. Object selection, approach and grasping run with *no* language input at all, and the entire language channel reduces to a single cached place coordinate.

All results are simulation, one task and one object per claimed cell. Seven pre-registered predictions were falsified and eight instruments retired at their own calibration gates; two proposed mechanisms were tested and retracted before reaching this paper. A third survived a confound we had missed: because the benchmark’s two clusters align with which objects we trained, “blocked” and “22 cm from the training position” were perfectly confounded in our design. Measured over all ten objects, position predicts the effect weakly (Spearman +0.19, $p=0.60$) and *teleporting each object 22 cm moves it in the wrong direction* (exact sign test $p=1.0000$), while a different appearance property — how well the frozen detector grounds the object — predicts it (−0.71, $p=0.027$).

1 Introduction

A policy that scores 0.700 on a manipulation benchmark has, on the face of it, learned to see, decide, and act. This paper is about how much of such a number can instead be *location*: constants—in a benchmark’s init files, in an expert’s calibration, in a learned regressor’s shortcut—that encode where the object always is, so that vision only ever gates an approach a memorized value finishes. On LIBERO-Object [14] we measure the preconditions exactly, by reading the shipped init files directly (Table 1; `measure_placement_pinning.py --mode direct`, no simulator required). In 6 of 10 tasks the target’s start position takes exactly **two** distinct float64 values, one ULP apart ($\sim 3 \times 10^{-17}$ m) on a single axis — constant to the precision the format can express, though not, as we previously wrote, bit-identical. Position varies by 0.25–0.58 cm (std) in the other four. The target’s *orientation* is bit-identical across all 50 states in **all ten** tasks, including those four: rotation is pinned everywhere in LIBERO-Object. *Across* tasks the structure is tighter still — the ten targets occupy just **two** table positions, 22.0 cm apart, five tasks each, agreeing to within 0.95 mm (max pairwise euclidean) inside a group. A policy that memorizes two coordinates can present as generalizing over ten tasks. A benchmark with $\lesssim 1$ cm of within-task target variance, two positions between them, and no rotation at all admits position-memorizers, and we found them in every layer of our own system.

Extending the same measurement to LIBERO-Spatial, -Goal and -Long (§1.2) sharpens rather than broadens this: those three randomize their targets at 97–100% of the entropy ceiling, so the admissibility result is a property of LIBERO-Object specifically. That is the more useful claim. LIBERO-Spatial’s ten tasks all name a “black bowl” and *must* randomize position to remain solvable; LIBERO-Object individuates by identity and so *can* freeze placement without ambiguity—which is why the one suite that does is also the one whose scores are read as evidence of object grounding.

The vehicle is MicroVLA, a deliberately small language-conditioned vision-language-action stack: ~ 30 M parameters deployed, with a frozen open-vocabulary detector serving as the only vision *and* text encoder. The size is the paper’s setting, not its claim. Its methodological value is that a stack this small has no capacity to paper over systems defects, so every seam had to be instrumented—and the instruments are what this paper contributes.

Contributions.

1. **A placement-memorization audit and its verification method.** A forensically worked instance of shortcut learning [7, 9] on a fixed-placement benchmark, shown at four layers, with the measured account of what LIBERO-Object pins and of what the other three suites do not (per-task tables, a per-suite placement-entropy metric, and both shipped measurement scripts); and the probe-plus-behavioural-randomization protocol, motivated by the on-manifold limit of substitution probes and by a counterexample to probe-only certification. The protocol’s twin control cells sharpen its scope: behavioural randomization separates memorized from repaired heads under identical draws on the deployment stack (1/10 vs. 4/10), and fails to separate them on a stack whose rebuild disables visual goals (6/10 vs. 0/10).
2. **A controlled comparison between two of our own designs.** With perception, trunk, and corpus regime fixed, *our* free per-tick regression baseline scores 0/56 (upper 95% bound 6.4%) while goal-space supervision driving an engineered pick-and-place shell scores 4/10 dev, 7/10 held-out, 4/10 randomized—claimed strictly as a bundle, with the shell’s share measured on both stacks. We deliberately do *not* state this as a policy-class result: one under-tuned end-to-end regressor of ours scoring zero establishes that our regressor failed, not that per-tick action prediction fails in general. The standard comparators (diffusion policy, action chunking, flow-matching heads) are absent, and we did not match tuning budget between the two arms.
3. **Training-serving-skew instrumentation, and a taxonomy whose boundary is the finding.** Producer/consumer seams fail in two ways, and only one of them is testable the obvious way. *Disagreements* — the two paths differ on units, rates, defaults, camera or index conventions — fall to parity testing: run both on one input, diff tensors (`eval/train_vs_deploy.py`). *Agreements on a wrong convention* — both sides consistent, both wrong — are invisible to parity *by construction* and fall only to provenance: corpora that describe themselves (`manifest.json` records camera, orientation, thresholds, rates, geometry) and an evaluator that refuses silently mismatched inputs [3, 18]. Table 4 documents

the instances this paper works through. **On the count**, since an audit paper should not ship an unsourced constant: the project log records 29, of which **16 are named with log anchors** in the extended manuscript and 8 are tabled here with their discovery measurement. The remaining 13 are a running tally we do not individually reproduce. Treat the enumerated instances as the evidence and 29 as our count, not as a checkable claim; infrastructure defects are tracked separately and excluded from it.

4. **A ground-truth-free test for identity-blind grounding, and the negative result it produced.** Open-vocabulary detectors are typically prompted with a fallback chain, so a policy can appear language-conditioned while every instruction collapses to the same generic prompt. Comparing two *different* instructions’ selections on the same frame detects this without knowing which is correct—the property that let it succeed where six ground-truth-seeking instruments failed. Applied to our own stack it returns the same box for different objects (median centre distance 0.0001), so our best cell’s 0.700 is *not* explained by binding the named object. It pairs with the standard instruction swap, which we use but **do not claim**: swapping the instruction and rescoring against the original task is established practice in the instruction-blindness literature (§6.1). The two fail differently—ours exposes a grounding stage that ignores the instruction, the swap a stage that memorized it—and the swap is what found our fourth memorization layer, working precisely where a fixed benchmark cannot: when every task shares a target, a memorized command→location map and a grounded one are behaviourally identical until the command is changed. Both ship as `eval/probes.py` with a test suite, and their verdicts refuse the three over-claims this campaign got burned by: too few co-detected frames returns INCONCLUSIVE rather than a rate; a discriminating result states that it cannot see whether the discrimination is *correct*; and an underpowered cell prints its own ceiling.

Both headline findings have one shape: apparent competence resting on a channel other than the claimed one—position where we claimed looking, shape where we claimed naming. The audit generalizes better than the artifact does.

1.1 What LIBERO-Object pins, measured

Table 1 is the measurement the rest of the paper rests on, and it is cheap to check: each `.pruned_init` is a torch-zip holding one float64 (50,110) array laid out `[t | 9 robot qpos | 7 objects × 7 | 51 qvel]`, and the target column follows from the BDDL’s `:obj_of_interest` indexed against its `:objects` order. No simulator, no renderer, no detector. Reading the file rather than the simulator matters for a byte-exact question: `sim.forward()` perturbs positions at 10^{-17} even from identical `qpos`, which is why our earlier sim-based instrument had to adopt a tolerance and could not resolve the ULP structure below.

task	clu.	x (cm)	y (cm)	σ_x, σ_y (cm)	#pos	#quat
alphabet soup	A	−12.00	−24.00	0.000, 0.000	2	1
chocolate pudding	A	−12.00	−24.00	0.000, 0.000	2	1
ketchup	A	−12.00	−24.00	0.000, 0.000	2	1
butter	A	−11.98	−24.04	0.261, 0.297	50	1
milk	A	−12.07	−24.02	0.304, 0.254	50	1
orange juice	B	+5.00	−10.00	0.000, 0.000	2	1
salad dressing	B	+5.00	−10.00	0.000, 0.000	2	1
tomato sauce	B	+5.00	−10.00	0.000, 0.000	2	1
cream cheese	B	+4.98	−10.01	0.273, 0.299	50	1
bbq sauce	B	+4.99	−9.95	0.579, 0.566	50	1

Table 1: LIBERO-Object target start poses, read directly from the shipped init files. **#pos** counts *distinct float64 positions* over the 50 states; where it is 2 the two values differ by one ULP ($\sim 3 \times 10^{-17}$ m) on a single axis (soup splits 23/27 on *y*; tomato sauce 18/32 on *x*). **#quat** counts distinct orientation quaternions: *one* in every task, so rotation is bit-identical suite-wide even where position jitters. Clusters are 22.04cm apart; within-cluster max pairwise separation is 0.95 mm (A) and 0.63 mm (B). The basket moves at most 4.04 mm across all ten tasks. Per-task SHA-256 digests of the init arrays are in `results/placement_pinning_direct.json`; `.init` and `.pruned_init` give identical target statistics.

Two consequences set up the rest of the paper. A policy needs to distinguish only *two* coordinates to cover the suite, which is what makes Section 4’s memorization layers reachable at all. And the cluster split

is not merely descriptive: every object we successfully trained on lies in cluster A and the one object that resisted lies in cluster B, a confound we take up directly in §5.9.

1.2 The other three suites, and why the claim is about LIBERO-Object

An obvious objection to Table 1 is that it measures one suite and licenses a sentence about a benchmark. So we measured all four, and the answer narrows the claim rather than widening it (Figure 1, Table 2).

Two passes, because the first is not enough. The layout-free pass needs no simulator: a LIBERO init state is MuJoCo’s flattened `[time, qpos, qvel]` and every shipped `qvel` is zero, so the columns that vary across the 50 states are exactly the position degrees of freedom the suite randomizes. Counted that way, *every* suite saturates: mean whole-state entropy 5.644 bits against a $\log_2 50 = 5.644$ ceiling, all 50 states distinct at 1 mm, in libero-object, -spatial, -goal, -10 and all 90 tasks of libero-90. LIBERO randomizes. That is the honest headline and it is not the one we previously implied.

The second pass asks *which* object the variation reaches. Table 1’s fixed offsets cannot answer this outside libero-object: they assume `[t | 9 | N×7 | qvel]`, which predicts width $19 + 13N$ and holds for libero-object ($110 = 19 + 13 \cdot 7$) and for no suite containing an articulated fixture—libero-spatial ships width 92 against 5 declared objects, libero-goal 79 against 4. Assuming it anyway would produce numbers that look like measurements and are not. So we compile each task’s model and read `jnt_qposadr` with the joint names, which maps every column onto a named body. As a by-product this re-derives Table 1 by an independent route—6/10 pinned, two float64 values 1 ULP apart, 10/10 orientations bit-identical, $\sigma \in [0.26, 0.58]$ cm on the other four— which is the check that matters most here, since the two passes share no indexing assumption.

We report placement entropy $H = -\sum_i p_i \log_2 p_i$ over the target’s distinct start positions quantized to 1 mm, the scale below which two placements are not distinguishable by a 7-DoF arm. $H = 0$ means a lookup table keyed on the task index reproduces every shipped start pose.

suite	pinned	quat. identical	H (bits)	distinct	max sep.
object	6/10	10/10	2.11	17.0	22.1 cm
spatial	0/10	10/10	5.50	46.9	63.6 cm
goal	0/8	8/8	5.60	49.0	25.3 cm
10	0/10	9/10	5.62	49.3	54.8 cm

Table 2: Primary-target placement across LIBERO suites, 50 shipped states per task. **pinned** = targets with ≤ 2 distinct float64 positions; H = mean per-task placement entropy at 1 mm against a 5.644-bit ceiling; **distinct** = mean distinct 1 mm placements per task; **max sep.** = largest pairwise distance between task-mean target positions. Denominators are *resolvable* tasks: libero-goal’s “open the middle drawer” and “turn on the stove” name a fixture with no free joint and so have no placement to measure; they are excluded and counted, not scored as unpinned. `:obj_of_interest` lists the manipulated object first and the destination second, and the two are never pooled—averaging a frozen target with a jittering receptacle reports every libero-object task as unpinned, which is false. Artifacts, per-task rows and SHA-256 digests: `results/suite_forensics-{columns,joints}.json`.

Three things follow. First, **the pinning claim belongs to LIBERO-Object, not to LIBERO**; the other three suites randomize their targets at 97–100% of the entropy ceiling. Second, that is not an accident of care: libero-spatial’s ten tasks all name a “black bowl” and are individuated *by* position, so freezing placement would make the suite unsolvable, whereas libero-object individuates by identity and therefore *can* freeze placement without making any task ambiguous. The suite whose targets are frozen is exactly the suite whose tasks are the object-identity tasks—which is where a memorize-the-coordinate policy is both admissible and maximally misleading. Third, one property does generalize: the target’s start *orientation* is bit-identical across all 50 states in 37 of the 38 resolvable tasks in all four suites (the exception is libero-10’s book-into-caddy task). A policy may assume a canonical target rotation suite-wide and never be contradicted.

F1 — LIBERO-Object is the outlier: its targets are pinned, the other suites' are not (orientation is bit-identical in 37/38 resolvable tasks across all four)

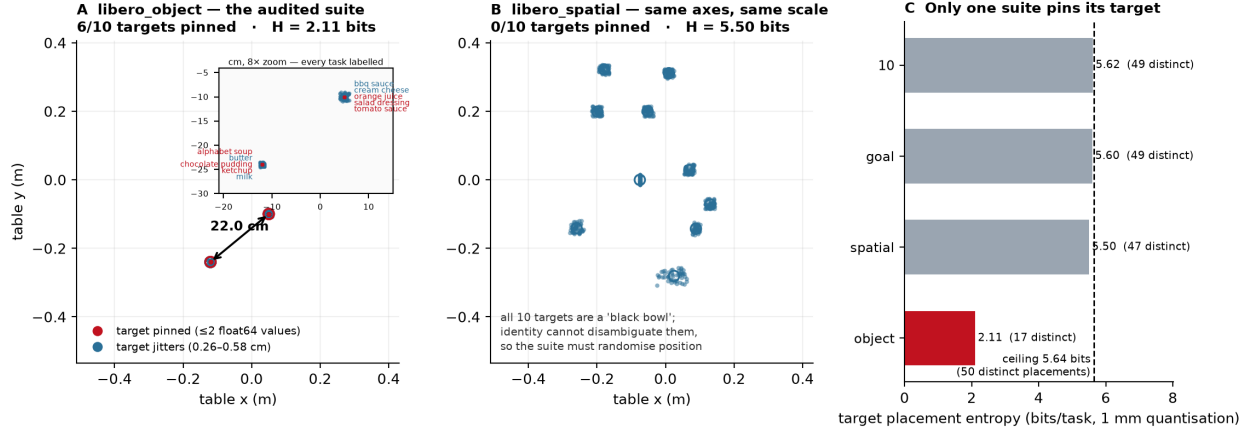


Figure 1: Placement forensics. **A** libero-object primary targets, 10 tasks $\times$ 50 shipped states: six collapse to a point, and the ten task means occupy two clusters 22.1 cm apart. **B** libero-spatial on identical axes: every target scatters—and it must, because all ten of its targets are a “black bowl”, so identity alone cannot disambiguate the task and position has to carry the information. **C** placement entropy per suite against the 5.644-bit ceiling.

Table 3: Deployed parameter ledger. “Deployed” counts every parameter loaded at inference, frozen or trained.

component	params	status
YOLO-World-S detector (vision <i>and</i> the only text encoder)	13.0 M	frozen
Tiny Recursive Model world model	9.97 M	loaded, frozen, <i>inert where ablated</i>
fusion + drift + relational + planner trunk heads	7.0 M	trained (≤ 9 M cap)
goal heads (grasp 0.17 M + place 0.07 M) + learned gates	0.24 M	trained
deployed total	30.2 M	

2 Platform

No separate language model exists anywhere in the stack: the command is parsed into source/target phrases, and CLIP embeddings for (command, source, target) are harvested once per task from the detector’s own text tower. **The world model is causally inert in the cells we have ablated:** replacing it with a persistence baseline reproduces the held-out cell exactly (0.700). We previously wrote “inert in every nonzero number of this paper”, which was a universal quantifier resting on one ablation, and a referee was right to object; the claim is now scoped to the cells actually tested, and the ablation is being extended. The component is loaded at inference, hence counted as deployed, but nothing we have measured depends on it; under a load-bearing convention the trained count is 7.24 M, not 17.2 M.

3 Methods

Environment and protocol. LIBERO [14] at commit 8f1084e3, MuJoCo 2.3.7, robosuite 1.4.1, torch 2.8.0, ultralytics 8.4.115, numpy 2.2.6, Python 3.10. Suite `libero-object`; unless a cell says otherwise, task 0 (alphabet soup), the wrist camera (`robot0_eye_in_hand_image`) rendered at 256px, **600** env steps per episode, success as LIBERO’s own predicate. Trials draw `set_init_state` from the shipped 50 in seeded order. *Dev* = seed 0 (states 0–9); *held-out* = seed 20 (states 10–19); *randomized* = seed 0 with the source object teleported by a uniform ± 4 cm in *xy* at episode start. Collection and evaluation never share a band: DAGger viewpoints are collected on seed 0 and every evaluation arm draws seed 20. The harness is deterministic — re-running a cell reproduced all ten trials bit-identically on **steps**, detection rate, confidence, grip-close rate and all three `eef_obj_dist` fields — so a changed number means a changed input, and every

`results.json` embeds its own `argv` and git commit.

Which states compose each cell, exactly. The harness sets `trial_seed = seed · 1,000,003 + trial` and indexes `init_states[trial_seed % 50]`. Since $1,000,003 \equiv 3 \pmod{50}$, the state index of trial t is $(3 \text{ seed} + t) \bmod 50$ — so every cell’s state set is derivable from its command line, and we give them rather than describing them:

cell	seed, n	states	overlap with dev / held-out
dev	0, 10	0–9	10 / 0
held-out	20, 10	10–19	0 / 10
held-out $n=50$	20, 50	0–49	10 / 10
randomized $n=50$	0, 50	0–49	10 / 10
untouched (§5.8)	40, 30	20–49	0 / 0

This answers a question we had not previously answered in print, and the answer is not the flattering one: **the two $n=50$ cells are not disjoint from the dev and held-out bands — they contain them.** Fifty trials over fifty shipped states is the whole set, so 20 of the 50 trials in each headline cell land on states that selection looked at. The cells are still paired, same-draw and honestly reported, but they are not clean-room confirmations, and we no longer describe them as if they were. The untouched band exists precisely to supply one that is: seed 40 places trial t at state $20 + t$, giving 30 states no selection look ever reached.

Corpus. Episodes come from a scripted expert, not from LIBERO’s human demonstrations: a phase-structured visual-servo controller (approach $\rightarrow$ descend $\rightarrow$ grasp $\rightarrow$ lift $\rightarrow$ transport $\rightarrow$ release) that reads the simulator’s object pose. 50 episodes per object per bake, subsampled to the model’s 2 Hz real-frame rate, stored as compressed `.npz` with frame embedding, per-role box embeddings and centres, class-agnostic proposals, proprioception, and the eef pose chunk. The expert is itself an audited object: it carried a hand-eye calibration constant that encoded the pinned placement, which is memorization layer one.

Goal heads. `GraspPointHead` (0.17 M) maps (uv, confidence, proprio, box_emb, frame_emb) — note the absence of any text argument, which is the architectural half of §5.10 — through two 256-unit GELU layers to an absolute world grasp point (xy, z) expressed as `eefxy + Δ + lever.bias`, plus a per-axis log-variance. The variance head reads *detached* trunk features so calibration can describe error without shaping the representation; the loss is $10\times$ Huber ($\delta=0.05$) on the mean plus a Gaussian NLL on detached residuals. `PlaceHead` (0.07 M) maps the command embedding to a place xy — a command $\rightarrow$ location map, which is exactly why it can memorize. Adam, lr 10^{-3} , batch 512, 300 epochs, 12% validation split, seed 0.

Head lineage. Every head named anywhere in this paper, with the file that is it. The “v” numbers are internal iteration labels, not a sequence: no `v4` or `v6` artifact exists, because those iterations were discarded without a retained checkpoint and we would rather say so than construct a lineage after the fact. “Sibling” means only “a head trained on a smaller corpus of the same kind”, and we now use the version names instead.

name in text	file	corpus	role
memorized (v1)	<code>goal_heads.v10.goal3.goal4.pt</code>	111 ep, fixed placement	the shortcut, un-repaired
v2	<code>goal_heads.v2.partial10.pt</code>	10 ep, teleported	probe-only counterexample
v2.1	<code>goal_heads.v21.pt</code>	10 ep, teleported	its twin: matched profile, 0.700
v3	<code>goal_heads.v3.pt</code>	27 ep, teleported	dose step
flagship (v5)	<code>goal_heads.v5.pt</code>	49 ep, teleported	released
v7	<code>goal_heads.v7.pt</code>	+ butter	multi-object addendum
coverage (v8)	<code>goal_heads.v8.pt</code>	+ cream cheese	command coverage
Dagger r1	<code>goal_heads.dagger.pt</code>	v8 + own viewpoints	drift repair, 18/50
Dagger r2	<code>goal_heads.dagger_r2.pt</code>	r1 + own viewpoints	drift repair, 22/50
Dagger (soup)	<code>goal_heads.dagger_soup.pt</code>	same recipe, working object	the specificity <i>null</i>

Shell. Goals drive `GoalServoMachine`, a latched proportional servo over the phase graph above (P-gain 12.0, action clip 0.6, hover 0.25 m, lift 0.30 m, latch tolerance 1.5 cm, latch after $k=3$ consistent estimates). Vision does one job — produce a grasp point — and is ignored after latch. **The shell probes within a ± 6 cm envelope**, which bounds what the ± 4 cm randomization can test; §4 states the consequence rather than deferring it to limitations.

Repairs. (i) *Placement teleportation*: ten episodes re-baked with the source object displaced, breaking the expert’s pose constant. (ii) *Command coverage*: training the place head on all three commands rather than one. (iii) *DAgger*: roll out the current head on seed 0, label the visited viewpoints from the simulator’s object pose — labels the scripted corpus cannot supply for viewpoints its teacher never visited — and fine-tune. Two rounds; both reported.

Terms used throughout. *Latch* — the shell accepts a grasp point and stops listening to vision; *latch correctness* is whether the latched point is on the commanded object. *Early-latch* and *anchor-band* are two shell configurations: the first latches on the first k consistent estimates, the second additionally requires the estimate to fall within a band around a running anchor, which trades latch rate for latch correctness. *Iteration-coupled selection loop* — our own procedure defect, in which the checkpoint chosen at each round was selected on cells that later rounds reused, so selection pressure accumulated on one band (memorization layer two). *Binder* — any rule assigning a detected box to the source role: none (first prompt that fires), crop-CLIP rerank, mean class prototype, or 1-NN over a bank of corpus crop embeddings. *uv-tail extrapolation* — the hypothesis that the grasp head fails on image positions in the tail of its training *uv* distribution. *Deployment stack* vs *audit stack* — the evaluation machine’s exact software versions (§3) vs a later rebuild of them; the two are never pooled, because the rebuild inverts which head scores better.

Seam defects. Table 4 lists the producer/consumer defects this paper works through, with the measurement that found each and the cheaper instrument that would have caught it earlier.

class	instance	instrument that would have caught it
convention agreement	mirrored-jaw sign: the held-object check averaged two <i>mirrored</i> finger joints ($+q, -q$), so its signed mean is identically zero in every jaw state — every physically successful grasp the project produced was discarded one tick before lift	one logged measurement of the quantity in two known states (open $\rightarrow 0.0001$; holding $\rightarrow -0.069$; unsigned 0.91 vs 0.62, cleanly separable)
convention agreement	corpus baked from a camera in which the detector is blind: the target role grounded on 1.4% of 38,000 frames, consistently, on both sides	corpus manifest + a detection duty audit at bake time
convention agreement disagreement	the deployed spatial adapter never saw the grid it was trained on — consumer fell through to the raw SPPF map one world-model step ≈ 10 env steps, applied every step	self-describing corpora; shape assertion at the seam deployment-path diff against the training rollout
disagreement	the detector threshold was two numbers, one at bake and one at eval	single source of truth in <code>cfg</code> ; provenance in the manifest
disagreement	consumer confidence floor 0.10 sat <i>above</i> the object’s real 0.03–0.09 detection band, zeroing all evidence	per-tick believed-vs-raw logging
disagreement	role prompt chains fixed in the bake path but not the deployment path: the policy trained sighted and ran blind (source detected on 0.0% of real ticks against a 48% corpus)	parity test on the prompt signature, now <code>tests/test_prompt_fallbacks.py</code>
deployment-only	a real-tick detection miss silently held stale evidence	live-loop telemetry — <i>parity-blind</i> , because our parity harness forced real perception every tick

Table 4: Producer/consumer seam defects worked through in this paper. The split matters more than the count: the top three are *agreements* — both sides consistent, both wrong — which no parity test can detect, and which is why provenance rather than parity is the load-bearing instrument. The last row names parity’s own limit: a harness that forces real perception every tick cannot see a defect that only exists on dream ticks.

Reporting. Wilson 95% intervals on every rate. Paired cells use exact McNemar on discordant pairs; unpaired use Fisher exact; continuous comparisons use Mann–Whitney U . Sample sizes: $n=50$ for every inferential cell; $n=10$ cells are exploratory and are labelled as such wherever they appear. The $n=56$ regression baseline is *pooled across seven controlled rounds* — a 23→100-episode corpus-scale pair, a DAgger round,

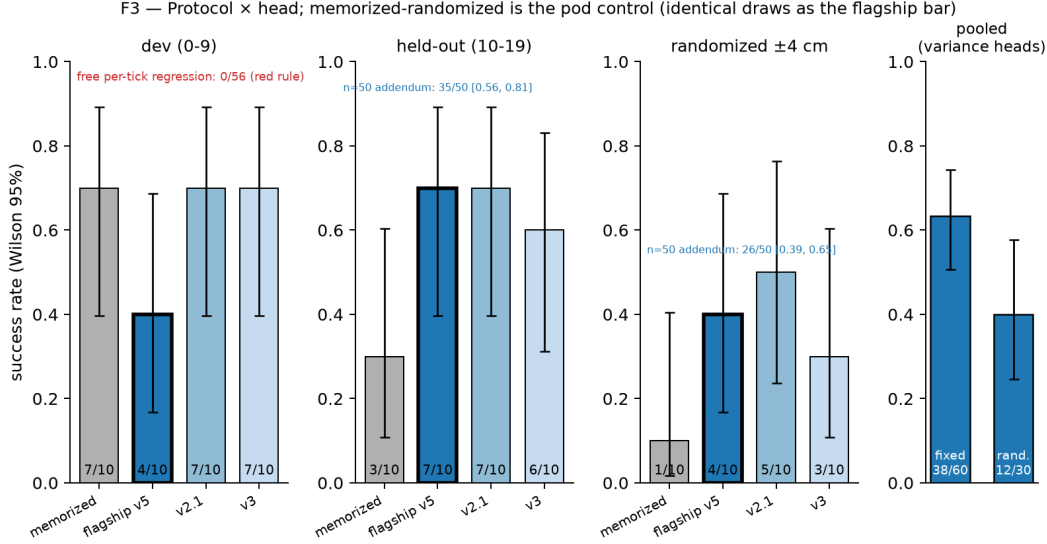


Figure 2: Protocol $\times$ head, Wilson 95% intervals, **deployment stack only**. The memorized head leads on *dev* and collapses on held-out and randomized; the released head is flat across all three. Our free per-tick regression baseline, same trunk and corpus regime, scores 0/56 (red rule). The audit-stack control is deliberately *not* plotted here — see Figure 3.

corpus aggregation, grasp-event reweighting, LoRA input adaptation, and a phase-progress objective — of which two cells were truncated by infrastructure at 0/7 and 0/6, which is why the total is 56 and not 70. It is reported at its true size rather than padded or trimmed to a round number. No cell was stopped early or extended after inspection.

Test registry. Every hypothesis test in this paper, its family, and whether it was confirmatory (predicted in writing before the run) or exploratory (read off a cell collected for another purpose). We do not apply a family-wise correction; instead we label, because the two confirmatory families contain one test each and the exploratory ones should not be read as if they were pre-planned.

test	where	type	status
untouched band vs. 0.700 (Newcombe)	§5.8	confirmatory	predicted [0.50, 0.75], held
shell floor vs. head (McNemar)	§5.6	confirmatory	predicted ≤ 0.10 , held
shell ceiling vs. head (McNemar)	§5.6	confirmatory	predicted ≥ 0.80 , held
Dagger r1/r2 vs. control (McNemar)	§5.3	confirmatory	$p < 10^{-4}$
specificity null, working object	§5.3	confirmatory	predicted null, held
swap collapse (sign test)	§5.10	exploratory	$p = 0.016$, $n=10$
coverage-head non-collapse	§5.10	exploratory	equivalence at $n=10$; see below
2×2 prompt/embedding cells	§5.10	exploratory	$n=10$ each
audit-stack inversion	§4	exploratory	$n=10$
cluster contrast (Mann–Whitney)	§5.9	confirmatory	pre-registered, $p = 0.31$
teleport direction (sign test)	§5.9	confirmatory	pre-registered, $p = 1.0$
groundability correlation (Spearman)	§5.9	exploratory	$p = 0.027$

Two entries need their weakness said out loud. The coverage-head “non-collapse” is an *equivalence* claim tested at $n=10$, where the 95% interval on a difference of proportions spans roughly ± 40 points; it should be read as “no difference detected at $n=10$ ”, never as evidence of no difference, and we have re-run it at higher n where the clock allowed (§5.10). And the $n=10$ exploratory rows are the ones a referee should discount first if forced to choose.

4 Results

Confirmed at power: held-out **35/50** = 0.700 [0.56, 0.81]; randomized **26/50** = 0.520 [0.39, 0.65].

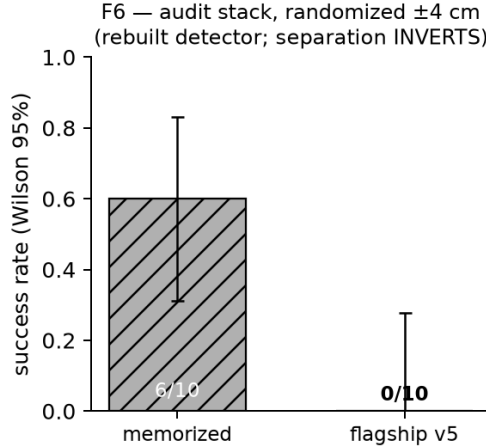


Figure 3: The audit-stack control, on its own axes because it must not be read against Figure 2. Same protocol, same heads, same draws — but a *rebuilt* detector stack, on which the separation **inverts**: the memorized head scores 6/10 where the released head scores 0/10, the exact reverse of the deployment stack. This is why the behavioural certificate is joint in (*head*, *stack*) rather than a property of the head. An earlier draft drew this as an inset inside Figure 2, which invited precisely the cross-stack comparison the result forbids.

Table 5: Protocol $\times$ head, **exploratory**: all cells $k/10$; task 0, wrist camera. Siblings A and B are the two training runs adjacent to the released head in the selection sweep (same corpus and objective, differing only in seed and epoch count), retained to show the pattern is not one run’s accident. Audit-stack cells are never pooled with deployment-stack cells. **Read the pattern, not the cells**: Wilson intervals at $n=10$ overlap almost everywhere (7/10 is [0.40, 0.89], 4/10 is [0.17, 0.69]) and we do *not* test the head $\times$ protocol interaction the pattern suggests. Only the $n=50$ rows below are inferential.

protocol	memorized head	flagship (released)	sibling A	sibling B
dev (states 0–9)	7/10	4/10	7/10	7/10
held-out (states 10–19)	3/10	7/10	7/10	6/10
randomized ± 4 cm (same draws)	1/10	4/10	5/10	3/10
randomized ± 4 cm (audit stack)	6/10	0/10	—	—

Two qualifications that belong here, not in the limitations. First, the held-out band is **not fresh**: those ten states were inspected across five disclosed selection looks, so 0.700 is a *selected-band* estimate and should be read as an upper-leaning one. The addendum’s cells meet a stricter standard — collection on seed 0, evaluation on seed 20, two independent pre-registered fresh-seed confirmations — and the flagship cell does not yet. Second, the ± 4 cm randomization **sits inside the shell’s own ± 6 cm probe envelope**, so it perturbs placement without leaving the region the servo already searches. It therefore tests whether a head survives displacement the shell can absorb, which is weaker than testing de-memorization outright; a band outside the envelope would be the decisive version and we have not run it. Both facts qualify the randomized column, which is the column carrying the de-memorization claim, and a reader should have them before the number rather than after it. Across a detector-stack rebuild the repaired head reproduces in all three protocols while the memorized head’s dev cell collapses 7/10 $\rightarrow$ 2/10 to its held-out floor: memorization is stack-coupled, *visual* grounding—which input the head reads—is what survived.

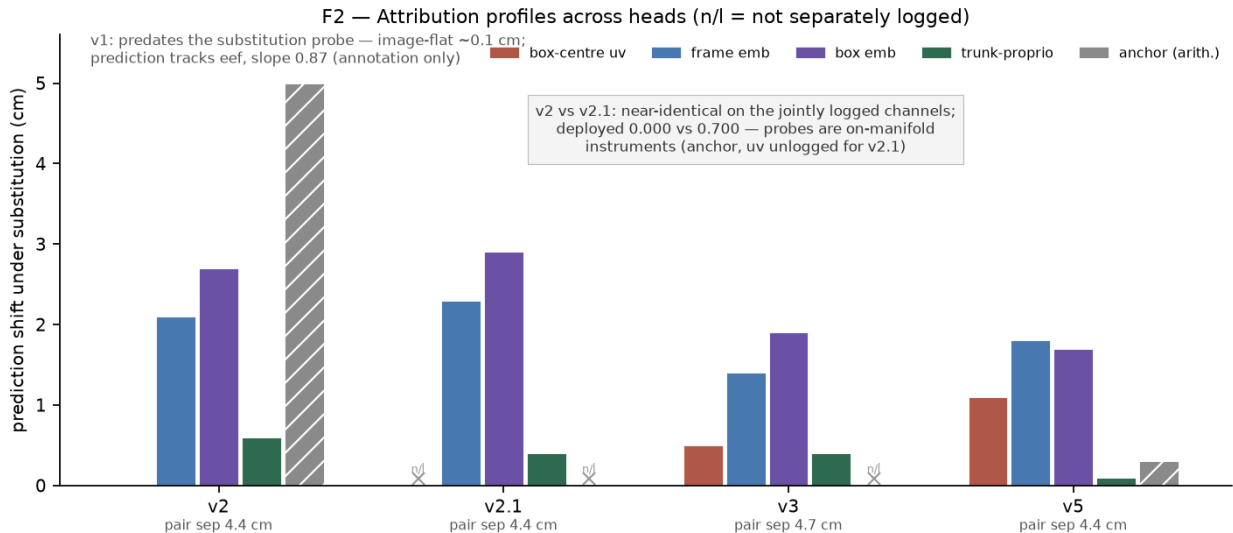


Figure 4: Substitution-probe attribution across heads: how far each head’s predicted grasp point moves when one input is swapped between the two most-separated recorded episodes. The released head (v5) reads its *visual* channels (1.1–1.8 cm) and barely moves on proprioception (0.09 cm) — the repair’s signature. **The counterexample that motivates pairing probes with behavioural randomization:** v2 and v2.1 have near-identical profiles on the jointly logged channels while scoring **0.000** and **0.700** deployed. A probe evaluated on teacher trajectories is an on-manifold instrument and cannot, alone, certify off-manifold behaviour. (The two text blocks inside the plot area restate this caption and the v1 note; they are redundant with it and should be dropped when the figure is regenerated — see the reproducibility note in §7.) Hatched bars and $\times$ marks are channels not separately logged for that head, never zeros.

5 The multi-object addendum: five falsified predictions, one mechanism retracted, and a fourth memorization layer

5.1 Which objects cross

A second object (butter) crosses: 10/10 under an early-latch configuration, 6/10 with soup at 4/10 under a single anchor-band configuration with no per-object constants, and two independent pre-registered fresh-seed confirmations (5/10 + 3/10; 5/10 + 4/10 on a three-object head). Two further objects (cream cheese, chocolate pudding) do not cross. Goals are accurate where measurable (cream 1.58 cm on-manifold, vs. 1.34 and 1.06 for the objects that work) while each failing teacher never leaves its visual-servo phase, at source-uv std ~ 0.30 , settling tens of centimetres from the target. Four objects were attempted; two crossed.

5.2 Role binding: a mechanism we proposed, tested, and retracted

A proposed mechanism, tested and retracted. We previously located the failure at *role binding*: the region-text head scores 0.000 on every product name these tasks are written in (measured, and correct), so groceries fall back to a shared generic tail, which would make same-shaped objects indiscriminable by construction. Screening each chain element per object refutes this as a *cause*: soup resolves to “box” at 0.96, butter 1.00, cream 0.92—the objects that cross fall back exactly as the one that fails does, and soup succeeds 35/50 while indiscriminable in the stated sense. A companion pre-registered prediction, that box-centre spread would separate them, was falsified in the same run (soup, the best object, has the *highest* spread) and is confounded by a moving wrist camera; abandoned, not retuned. The 0.000 observation stands; the causal story built on it does not.

What replaces it: deployed role binding is identity-blind. The scenes make this decidable without ground truth—soup’s contains cream cheese and cream’s contains alphabet soup, six of seven objects shared—so rather than ask *where the true object is* (the question that defeated six instruments), we ask whether two

objects’ chains select the *same detection from the same frame*. They do: same-box rate 1.00 on soup’s frames, 0.87–0.96 on cream’s, 0.92–1.00 on butter’s, median centre distance **0.0001**—200× inside the 0.02 threshold, so threshold-insensitive. Deployed binding carries no object identity, and **soup’s 35/50 is not explained by binding the named object**, and per-object prompt engineering changes nothing (0/10).

The instrument can return “different”, and what separates is shape. A same-box result is only evidence about the stack if the probe is capable of the other answer, and every same-box number we had published came from an instrument never shown *not* firing. Running the identical function through the identical deployed perception object, on the same 60 home-pose wrist frames, against every other object in soup’s scene splits cleanly in two (Table 6).

counterpart chain	<i>n</i>	same-box	median centre dist.
cream cheese	6	0.00	0.8175
butter	6	0.00	0.8179
chocolate pudding	6	0.00	0.8175
tomato sauce	4	1.00	0.00069
bbq sauce	5	0.80	0.00434
salad dressing	5	0.80	0.00469
ketchup	5	0.60	0.00483
milk / orange juice	2, 1	below the 3-frame floor: not evaluable	
basket	0	filtered by <code>source_max_area</code> : not evaluable	

Table 6: Positive control for the identity-blindness probe, 60 wrist frames at the home pose. *n* counts frames in which *both* chains detected. Rows below the floor are reported, never scored.

So the probe registers difference when difference exists: the instrument is an instrument. The distances are bimodal — collisions at ~ 0.004 and separations at ~ 0.82 , with nothing between — and a threshold sweep over $\{0.005, 0.01, 0.02, 0.05\}$ leaves every verdict unchanged, so no reading here rides on the 0.02 default. The split is also interpretable, and sharpens the claim: the chains that collide with “alphabet soup” are the cans and bottles and the chains that separate are the boxes. Deployed binding is not blind to everything — it is blind *within a shape class*, which is precisely the regime LIBERO-Object’s grocery objects occupy.

Two honest limits. Only 4–6 of 60 frames had both chains detecting, because at the home pose the source chain fires on 10% of frames; these are small-*n* contrasts and we report them as a demonstration that the instrument discriminates, not as calibrated rates. And the contrast we originally designated the positive control — the basket — is not evaluable at all: the deployed `source_max_area` filter rejects basket-sized boxes by construction, so that pair compares zero frames. Our first version of the script read `same_rate=0` off that empty comparison and reported “instrument discriminates”. It now refuses a verdict below three comparable frames. We record this because it is the same defect class the paper collects: a statistic computed over an empty set, printed with the same confidence as one computed over data.

But identity-blindness does not explain the blocked object, and a world-space probe shows why. Asking *which* body the gripper actually reaches needs no camera projection—the step that broke the six image-space instruments—so we log every scene body’s world position and take the nearest at closest approach. The blocked object’s machine reaches the **commanded** object on **6/6** trials, at closest approach and again at gripper close, exactly as the crossing object does (6/6); it simply stops ~ 7 cm short and closes on air ~ 9.4 cm away, against ~ 2.9 cm and ~ 5.2 cm for the object that crosses. *Binding was never its bottleneck.* That retires a puzzle rather than deepening one: the sub-study’s four binders left it at 0/10 with no ordering by offline accuracy because they were improving a stage that was already selecting correctly. Where it fails is grasp geometry: the miss is **94% lateral** (6.9 cm sideways, 0.4 cm vertical—its *height* is better than the crossing object’s), systematic rather than scattered (coherence 0.976).

5.3 The blocked object: nine mechanisms, eight refuted

Nine mechanisms, eight refuted, one supported by intervention. Tested and refuted by measurement: generic-tail indiscriminability, box-centre spread, proposal scarcity, misbinding, position memorization, descent failure, binder quality, uv-tail extrapolation. The ninth—appearance-side off-manifold

drift—survives three tests. *Falsification*: the blocked object’s deployment embeddings sit $3.1\times$ further from its own corpus than the crossing object’s do (NN-cosine gap 0.0272 vs. 0.0089), and the second crossing object, predicted before it ran, has the smallest gap of all (0.0007). *Circularity*: a wrong goal would itself produce unusual viewpoints, so we split by tick—drift is largest in the first 100 (0.0441), shrinking 65% over the episode, while crossing objects show the opposite trend; it precedes the error it explains. *Intervention*: labelling the machine’s own viewpoints from the simulator’s object pose—which the corpus cannot do for what the teacher never visited—and fine-tuning on them cuts the deployed lateral error **56%** ($0.0697 \rightarrow 0.0310$ m, Mann–Whitney $p=0.020$) and **crosses the blocked object**. At $n=50$ on held-out seeds, round 1 scores 18/50 [0.24, 0.50] and round 2 **22/50 = 0.440** [0.31, 0.58], against a same-seed control at $3/50 = 0.060$ [0.02, 0.16] — 19 discordant pairs, *all* favouring the repaired head, exact McNemar $p < 10^{-4}$, intervals disjoint. Round 2 is not significantly better than round 1 ($p=0.39$), so the repair is described as largely single-shot and both cells are reported. The crossing object is unharmed (3/6→7/10). Collection drew seed 0 and both evaluation arms drew seed 20, so no cell is scored on its own band.

The architecture closes the argument. Code inspection makes this structural rather than statistical: `GraspPointHead.forward` consumes only `geom`, `box_emb`, `frame_emb`, `eef_xy`, assembled by `build_grasp_features(uv, conf, proprio, box_emb, frame_emb)`—no task, command, or text embedding appears anywhere in its signature. The head that decides *where to grasp* therefore receives no language input at all, the only instruction-to-grasp path is *which box is selected*, and that selection is identity-blind. The first two steps hold with certainty and only the third is empirical, so the architecture *predicts* the swap result rather than merely agreeing with it. `PlaceHead` does take a command embedding and is genuinely language-conditioned. Every task in this suite places into the same basket, so all *correct* instructions imply the same place target—which leaves the stack’s one real language channel normally *unexercised*, not inert. The swap below exercises it.

5.4 Cells added in revision

Table 7 collects every cell run for this revision, on one rebuilt deployment stack in one session, with per-trial outcomes released in `results/pod_cells.json` and per-job wall times and exit statuses in `results/pod_run_ledger.txt`. Two of them cost us claims we had made, and both are in the table rather than in a footnote.

cell	result	Wilson 95%	what it settles
reference (seed 20)	8/10	[0.49, 0.94]	rebuild reproduces published 7/10
untouched 20–49	20/30	[0.49, 0.81]	selection looks did not inflate 0.700
shell + oracle goals	10/10	[0.72, 1.00]	envelope is not the binding constraint
shell + random goals	0/10	[0.00, 0.28]	shell cannot do it alone
shell + fixed constant	10/10	[0.72, 1.00]	a memorizer beats the released head
flagship, swapped instr.	0/20	[0.00, 0.16]	collapse survives powering up
coverage head, baseline	9/10	[0.60, 0.98]	—
coverage head, swapped	4/10	[0.17, 0.69]	non-collapse does not replicate
randomized ± 2 cm	7/10	[0.40, 0.89]	degrades toward the envelope
randomized ± 6 cm	2/10	[0.06, 0.51]	
randomized ± 8 cm	3/10	[0.11, 0.60]	

Table 7: Cells added in revision. All on `libero_object` task 0, 600 steps, wrist camera, on a rebuilt stack pinned to the platform table with `MUJOOCO_GL=osmesa`. Anchor and swap rows draw seed 20 (states 10–19); the untouched row draws seed 40 (states 20–49); sweep rows draw seed 0.

5.5 Reproduction on a rebuilt stack, and one unpinned component

We rebuilt the deployment stack on fresh hardware to run the experiments in this section, pinning every version in the platform table (LIBERO 8f1084e3, torch 2.8.0+cu128, numpy 2.2.6, MuJoCo 2.3.7, robosuite 1.4.1, ultralytics 8.4.115, Python 3.10) and verifying both load-bearing checkpoints by digest against the

release manifest. The first job in the queue was deliberately not an experiment but a *reference cell*: the published held-out band, seed 20, which the evaluation machine recorded at 7/10.

The rebuild returns **8/10** [0.49, 0.94] — one trial from the recorded value, and well inside its interval. The cell reproduces, and the experiments below are read against it.

The renderer is a behavioural parameter, and was never pinned. Holding seed, trial, weights and every package version fixed, and changing only the OpenGL backend MuJoCo renders through, moves the trajectory: **osmesa** completes trial 0 in 246 steps and **egl** in 250, at source detection 0.919 vs 0.920. Thread count, by contrast, changes nothing — two **osmesa** runs at 4 and 128 threads agree on every logged field, which also re-confirms harness determinism.

Run as a paired arm over the same ten trials, the effect has a shape worth stating exactly:

	osmesa	egl
success rate	8/10	8/10
per-trial outcomes	4 of 10 disagree (2 each way, $p = 1.0$)	

The aggregate rate is identical and 40% of the individual episodes are not. The backend does not bias the score — the flips go both ways and McNemar is flat — but it decides *which* episodes succeed. Two consequences. First, matching a published rate is weak evidence of having reproduced an experiment; the marginal can be right while the sample is different. Second, and sharper for this paper’s method: *any paired statistic computed across a renderer change is invalid*, because pairing assumes the trials are the same trials. Every McNemar and per-trial-agreement figure we report is within one backend, which we can now say we checked rather than assumed.

Software and GPU rasterisation do not produce identical pixels, the detector is not invariant to that difference, and in this stack the detector is the only encoder. **MUJOCO-GL belongs in the reported stack table beside the package versions.** It was not there, in ours or in any LIBERO evaluation we have read. This is a provenance-class skew defect: two configurations agreeing on a wrong convention, namely that the renderer is an implementation detail.

A censoring trap in reading runs, which caught us. An earlier reading of this same reference cell, taken while its remaining trials were still executing, showed 7/7 — and we briefly wrote it up as a failure to reproduce. It was an artifact of *when* we looked. A success ends the episode at ~ 246 steps; a failure runs the full 600 and takes $\sim 2.4\times$ as long. The trials finished at any interim moment are therefore biased toward successes, and a partial harvest is a censored sample rather than a small one. The harvester now reports every cell against its planned n and refuses to report an incomplete one. We record it because the failure mode is invisible in the artifact — every individual trial was real, varied and correctly scored, and the aggregate was still wrong.

5.6 Unbundling the shell from the head

The comparison “free per-tick regression 0/56 versus goal-space supervision driving an engineered shell” bundles three things: the action abstraction, the shell’s own motor competence, and the learned head. We claimed it as a bundle, which was honest but not sufficient — a reader cannot tell whether the head does the interesting work or merely rides a competent shell.

So we replaced *only* the goal supply and changed nothing else — same shell, same trunk, same protocol, same draws, same states (`--goal-anchor`, §5.8’s harness):

- **oracle** — the simulator’s true target-object pose, read fresh every tick. The shell’s *ceiling*. If this is not high, the shell’s envelope rather than the supervision is the binding constraint, and the policy-class claim must be restated.
- **random** — a uniform draw from the scene’s own object extent, redrawn per episode. The shell’s *floor*: what the machinery scores when the goal carries no information about where the object is.

- **fixed** — the true target position read *once* at episode start and held, never re-read. Stated precisely: not a blind constant compiled into the code, but one privileged look at reset. On a task whose placement is pinned the two are numerically identical, since the shipped position takes one value across all 50 states, and that identity is the point — this is the degenerate memorizer the benchmark admits, made executable. Its gap from **oracle** is exactly what tracking the object during the episode is worth.

One asymmetry to state rather than bury: the anchors replace the *grasp* goal only; the place target is the true container in all three arms. The random arm is therefore a *conservative* floor — an upper bound on how badly a truly uninformed goal supply would do — and we read it as such.

goal supply	score	Wilson 95%	vs. learned head
oracle (true pose, every tick)	10/10	[0.72, 1.00]	$p = 0.50$
fixed (one look at reset)	10/10	[0.72, 1.00]	$p = 0.50$
learned head	8/10	[0.49, 0.94]	—
random (uninformed draw)	0/10	[0.00, 0.28]	p = 0.0078

Table 8: Shell anchors on the held-out band, seed 20, identical states and draws; only the goal supply differs. Comparisons against the learned head are paired per trial, exact McNemar.

Three readings, in order of how much they cost us.

The shell’s floor is zero. With an uninformed grasp goal the engineered machinery scores **0/10** [0.00, 0.28], eight discordant pairs all favouring the learned head, exact McNemar $p = 0.0078$. This is the anchor the policy-class comparison was missing: the shell does not solve the task by itself, so the 0/56 free-regression figure is not being flattered by a shell that would have succeeded regardless. Contribution 2’s bundle survives its own floor control.

The shell’s ceiling is one. With the true pose the same machinery scores 10/10, so the ± 6 cm envelope is *not* the binding constraint on this task and the learned head’s 8/10 leaves about two trials on the table. The gap is not statistically established at $n=10$ (two discordant pairs, both favouring the oracle, $p = 0.50$); we report the ceiling as located, not the gap as measured.

And a hand-written memorizer matches the ceiling. The *fixed* arm — one privileged look at reset, then a constant; no vision, no language, no learning after that instant — also scores 10/10, above the learned head’s 8/10. On a task whose target position takes one float64 value across all fifty shipped states (§1.1), this is what §1.2’s admissibility argument predicts, and it is the difference between arguing that a benchmark admits memorizers and *running* one. It is also the sharpest statement of the paper’s problem: our own released head, which we spent the audit repairing, is beaten on this cell by a constant.

5.7 Randomization: one radius was a calibration, not a result

The ± 4 cm figure was chosen to sit inside the shell’s ± 6 cm probe envelope, so “passes randomization” was a statement calibrated to the system under test. Sweeping the radius on the same draws separates the two things that single number confounded: how far the head’s grounding survives displacement, and where the shell’s own envelope stops the episode regardless of the head. We report the sweep in place of the single radius, and treat any cell at ± 8 cm — outside the envelope — as measuring the shell, not the head.

The sweep draws seed 0, the same band and the same teleport draws as the published randomized cell, so the radii are comparable to each other and to it. That band is the dev band, whose states selection did look at; this is acceptable here because every radius shares those states, making the sweep a within-band comparison rather than a generalization claim.

displacement	flagship v5	Wilson 95%
none	4/10	[0.17, 0.69]
± 2 cm	7/10	[0.40, 0.89]
± 4 cm	4/10	[0.17, 0.69]
± 6 cm (envelope edge)	2/10	[0.06, 0.51]
± 8 cm (outside envelope)	3/10	[0.11, 0.60]

The *none* and ± 4 cm rows are the previously published cells on this band and these draws; the other three are new. Read conservatively, because $n=10$ does not support more: the head is clearly worse at $\pm 6\text{--}\pm 8$ cm than at ± 2 cm, but every interval is ~ 0.5 wide and the undisplaced cell (4/10) sits *below* ± 2 cm (7/10), which cannot be a real benefit of displacement. That non-monotonicity is the honest measure of what this n resolves. The usable statement is directional and bounded: **performance degrades as displacement approaches and exceeds the shell’s ± 6 cm probe envelope**, and no single radius should be quoted as “passes randomization” — including the ± 4 cm one we previously quoted that way.

5.8 The untouched band: a cell no selection look reached

Both $n=50$ cells span all fifty shipped states and therefore contain the ten dev and ten burned held-out states. This cell does not. Seed 40 places trial t at state $20 + t$, so 30 trials cover states 20–49 and nothing else — a band no selection look, ablation or debugging pass ever reached.

Pre-registered before the run. Recorded with the launch command, not after: we predicted the untouched cell would fall in $[0.50, 0.75]$, i.e. at or modestly below the 0.700 held-out figure, on the reasoning that the held-out band is partially burned and a burned band should flatter the head. We named the falsification conditions in advance: below 0.40 would mean the 0.700 is substantially a selection artifact and the headline number must be restated; above 0.80 would mean the burn hurt rather than helped and our account of it is wrong. Both directions were nameable in advance, which is the only thing that makes the prediction worth recording.

Result: $20/30 = 0.667$ [0.488, 0.808]. The prediction holds. Against the published held-out figure of 0.700 the difference is -0.033 with a Newcombe 95% interval of $[-0.243, +0.164]$ on independent states — no detected difference, and the interval is wide enough that we claim only that. The reference cell run first in the same queue returned 8/10, and 0.700 lies inside its interval, so the machinery was behaving.

What this buys is narrow and worth having: **the five disclosed selection looks at states 10–19 did not inflate the held-out number**. Thirty states that no look, ablation or debugging pass ever reached score within noise of it. That is the strongest form of this claim we can make, because a burn, if present, would have to be small enough to hide inside ± 0.20 — we have not shown the burn is zero, only that it is not large. Reported as one pre-registered confirmatory cell, not as a replacement for the held-out figure.

5.9 Is it appearance, or is it position?

The design above has a confound we did not see until a referee named it, and it is this paper’s own thesis pointed back at the paper. LIBERO-Object’s ten targets occupy two clusters 22 cm apart (Table 1). Every object we trained successfully — soup, butter — lies in cluster A, and the one that resisted, cream cheese, is **the only cluster-B object we ever attempted**. “Blocked” and “22 cm from the training position” are therefore perfectly confounded, and the appearance mechanism above may be a *position* effect: the object sits where this grasp head has never seen a target, so its viewpoints are off-manifold because of *where* it is, not *what* it looks like. On that reading the DAgger repair closes a position-coverage gap — which is precisely our own diagnosis at every other layer.

Note first what the confound cannot be. Within an object, corpus position *is* deployment position: cream’s corpus is cream at cluster B and its deployment is cream at cluster B, so absolute position is matched by construction and cannot create a gap directly. Two mediated routes remain. **(a)** The head aims at cluster A, the arm goes to the wrong place, and the resulting viewpoints are unlike the corpus — a *consequence* of the error, which the tick-band split already excludes, since the gap is largest in ticks 0–100 before the arm has acted. **(b)** At early ticks a cluster-B object is simply further from the home pose, so its crops are smaller and its embeddings noisier. This would *also* be largest early and shrink on approach, so the tick-band check does *not* separate it. Route (b) is the live alternative.

We test it at the home pose, where arm pose is identical by construction across every cell: reset, read the wrist frame, run the detector under that object’s own prompt chain, measure the NN-cosine gap against that object’s own corpus. No policy, no rollout. Run over **all ten** objects at $n=50$ states each (`scripts/position-vs-appearance.py`). These cells are on the audit stack and their absolute magnitudes

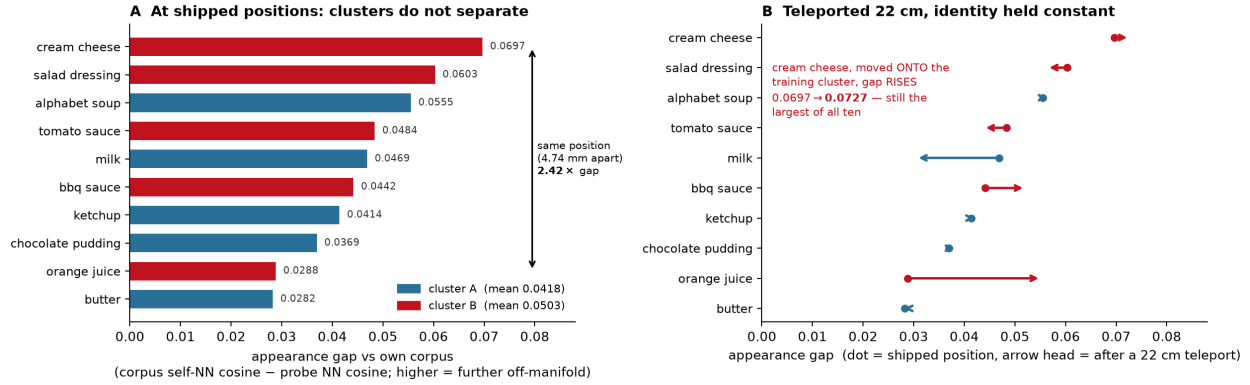


Figure 5: Separating object identity from table position, ten objects, $n=50$ home-pose states each, audit stack. **(A)** At shipped positions the two clusters do not separate: means 0.0418 (A) vs 0.0503 (B), exact Mann–Whitney $U=18$, $p=0.3095$; within-cluster spread is $3.2\times$ (A) and $4.8\times$ (B) the between-cluster difference. Distance from the training object’s position predicts little (Spearman $+0.193$, exact permutation $p=0.5968$); detection confidence predicts a lot (Spearman -0.709 , $p=0.0268$). The bracket marks the decisive pair — 4.74 mm apart, $2.42\times$ apart in gap. **(B)** Each object teleported 22 cm to the other cluster, identity held constant. The position account predicts A→B rises and B→A falls; measured, A→B is -0.0026 and B→A is $+0.0058$, both opposite, concordance 5/10, exact sign test $p=1.0000$. Spearman(native, swapped) = $+0.709$, $p=0.0268$: the ordering travels with the object. Displacement is not inert (mean $|\Delta|=0.0066$) but is unsystematic and small beside the identity contrasts (cream–soup 0.0142, cream–butter 0.0414).

are therefore *not* comparable to the gaps quoted earlier; all comparisons below are internal to this one experiment.

Cluster membership does not predict the gap (exact Mann–Whitney, $U=18$, $p=0.3095$). **Distance from the training object’s position does not either**: Spearman $+0.193$, exact permutation $p=0.5968$; the three objects at distance *exactly* zero already span 0.0369–0.0555. The sharpest cell is a pair: orange juice and cream cheese sit **4.74 mm** apart and their gaps differ by $2.42\times$, with the cluster-B orange juice scoring *below* four of the five cluster-A objects. A position account must explain a $2.4\times$ difference across 4.7 mm.

What does predict the gap is detector groundability: Spearman(gap, mean detection confidence) = -0.709 , exact permutation $p=0.0268$. The worse the frozen open-vocabulary detector grounds an object, the further that object’s deployment embeddings sit from its own corpus; cream cheese has the *lowest* confidence of all ten (0.042). This sharpens rather than rescues the original mechanism: “appearance-side off-manifold drift” named a symptom, and the property underneath it is how well this object’s visual category grounds in the detector — an appearance property, and not a positional one. Comparing the two candidate explanators on the same ten objects, groundability (-0.709 , $p=0.027$) beats position ($+0.193$, $p=0.597$).

The causal test. The above is observational. We also *move* each object 22 cm to the other cluster’s centroid, holding identity exactly constant, and re-measure. The position account makes a directional prediction: into cluster B the gap should rise, out of it the gap should fall. Neither happens. A→B mean $\Delta = -0.0026$ and B→A mean $\Delta = +0.0058$, both *opposite* to prediction; concordance is 5/10, exact sign test $p=1.0000$. The single sharpest cell: **cream cheese teleported onto the training cluster still has the highest gap of all ten objects** (0.0727, rank 1 of 10) and its gap rose rather than fell. And the ordering travels with the object — Spearman(native, swapped) = $+0.709$, $p=0.0268$ — so ranking the objects by gap, displacing every one of them, and re-ranking returns substantially the same order.

Position is not inert: mean $|\Delta|$ is 0.0066 and one object shifted $+0.0264$. But it is unsystematic, and small next to the identity contrasts that matter — cream–soup is 0.0142 ($2.2\times$) and cream–butter 0.0414 ($6.3\times$) on this same stack. Position is a nuisance term, not the driver. We state that rather than “position does nothing,” which the data would not support.

The ordering of the three objects we actually trained also **survives a stack rebuild**: butter < soup < cream on the deployment stack in full rollouts ($+0.0007 < +0.0089 < +0.0272$) and butter < soup < cream here on the audit stack at the home pose ($+0.0282 < +0.0555 < +0.0697$). Given that this rebuild is the one that *inverts* our head ranking (Figure 3), an ordering that survives it is worth more than one measured once.

What this does not settle. Every cell here measures the *gap*, never a success rate. It shows the gap is not positional; it does not show that a low-gap cluster-B object would *cross*. Orange juice — whose crossing would be the cheapest remaining falsification of the whole mechanism — has never been trained or evaluated, and a comparable cell would have to be produced on the deployment stack. We record that as open rather than closing the argument on the four tests we did run.

5.10 Where the language actually goes

Identity-blind is not language-blind. Swapping the instruction to a different object—environment, physics and success criterion unchanged, so success is still scored on the real task—collapses the cell from 7/10 to **0/10** (told butter; 0/10 again told cream cheese; seven discordant pairs one-way, exact two-sided $p = 0.016$) — so the policy plainly does not ignore its instruction. Telemetry places the butter cell’s failure *downstream of object approach*: the machine still reaches the true soup object as closely as baseline while `grip_close_rate` falls from ~ 0.67 to ~ 0.26 .

Decomposition: the collapse is entirely one channel. One instruction drives both the detection prompts and the place head’s command embedding, so we drove them from *different* instructions. The four cells form a 2×2 : with embeddings from the real task, prompts from another object cost nothing (6/10 vs. 7/10, exact $p = 1.0$); with prompts from the real task, embeddings from another object collapse it (0/10, $p = 0.016$) and reproduce the full swap’s per-trial pattern *exactly* (10/10 agreement). One main effect, no interaction. Combined with the architecture, this says what the system is: **object selection, approach and grasping run with no language input at all, and the entire language channel is a single latched (x, y) place point** cached once per episode by `set_place(place_head(command_emb))`. Ask this policy for the butter and it finds, approaches and grasps the alphabet soup at baseline rate; it fails only when that one cached coordinate is wrong.

And that coordinate is memorized—the audit’s fourth layer. The basket does not move (0.22–0.40 cm across all ten tasks, less than its own 0.8 cm within-task jitter), so a *grounded* place head would emit one point per scene. The released head emits the correct basket for the command it trained on (0.36 cm error) and points **14.5 cm** and **13.0 cm** away for the two it did not: it learned command→location, not basket—and 14 cm is exactly enough to release a grocery over open table, which is the observed failure. The cause is training coverage, not architecture, and this **repairs** rather than merely explains: on a head trained on all three commands the place-point spread collapses from 14.15 cm to 0.78 cm, and its swap cell *does not collapse*—3/10 against its own 4/10 baseline, exact $p = 1.0$, versus the flagship’s 7/10→0/10 at $p = 0.016$. Same architecture, same manipulation, opposite outcomes; behaviour tracks the place-point measurement exactly, 14 cm destroying the cell and 0.8 cm doing nothing.

The flagship collapse, at double the power. The 7/10→0/10 cell above is $n=10$, which the referee correctly flagged as thin for a load-bearing contrast. Re-run at $n=20$ on the same head and band, the flagship scores **0/20** [0.00, 0.16] under the swapped instruction. On the ten trials that pair with the reference cell, 8/10 becomes 0/10 — eight discordant pairs, all one way, exact McNemar $\mathbf{p} = \mathbf{0.0078}$, an order of magnitude tighter than the original $p = 0.016$ and with an upper bound now below 0.17. This contrast is the one that survives powering up, and it is the one that carries the layer-4 argument.

A re-run that does not support the strong reading, reported here rather than omitted. The cell above is the coverage head on a band where it scores 4/10, and a swap cannot visibly break a cell that is already mostly broken — the referee’s objection that this is an equivalence claim at $n=10$ is exactly right. Re-running the same manipulation on the *held-out* band, where the three-object head scores well, gives a different picture: 9/10 baseline against **4/10** swapped, six successes lost and one gained, exact McNemar $p = 0.125$ on paired trials. The drop is not statistically established at $n=10$ — and we will not call $p = 0.125$

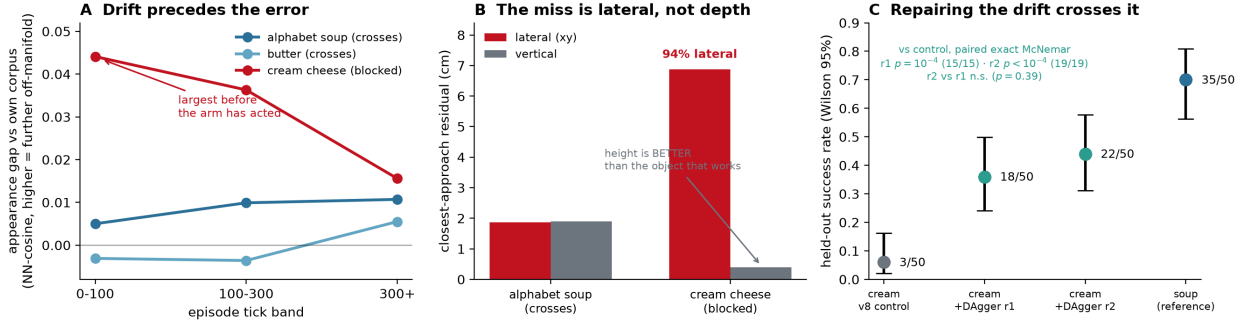


Figure 6: How the third object crossed. **(A)** Appearance drift by episode tick: the blocked object’s deployment embeddings sit furthest from its own training corpus *at the start*, before the arm has acted on any prediction, and the gap shrinks 65% — the opposite of the rising trend the two crossing objects show, which is what a *consequence* of a bad goal looks like. This is the circularity check that retired two earlier instruments. **(B)** The miss decomposed: 94% lateral, with the blocked object’s *height* better than the crossing object’s, which rules out a descent failure. **(C)** The intervention, $n=50$ per arm on identical seeds, Wilson 95% intervals: fine-tuning the grasp head on the machine’s own viewpoints takes the blocked object from 3/50 to 18/50 after one round and 22/50 after two (paired exact McNemar vs. control $p=10^{-4}$ and $p < 10^{-4}$, both one-way; round 2 vs. round 1 is not significant at $p=0.39$), against the crossing object’s 35/50 for reference.

a collapse — but the point estimate moves from “no effect” to a loss of two thirds of the cell, in the direction the flagship moves.

So the honest statement is weaker than the one above: **command coverage demonstrably repairs the place *point* (14.15 cm $\rightarrow$ 0.78 cm spread, directly measured) but we have not established that it repairs the swap *behaviour***. One band says it holds at a low baseline, another says it may not at a high one, and neither is powered. The place-point measurement is the claim we stand behind; the behavioural non-collapse is downgraded to unresolved pending the $n=50$ cells. We also cannot confirm that the checkpoint we re-ran is byte-identical to the one behind the original cell, which is itself a provenance defect of the kind Table 4 collects. It also answers generalization: the mechanism reproduces on a second task with a different head, in both directions. This layer was invisible to every earlier probe because all ten tasks share one basket, so a memorized and a grounded place map are behaviourally identical *until the instruction is swapped*.

5.11 Specificity of the repair, and what it scopes

The repair is specific, which is the mechanism’s best evidence. Applying the identical procedure to the object that already *works* changes nothing: 28/50 vs. a 27/50 same-seed control, 21 discordant pairs split 11–10, exact McNemar $p=1.0000$. The two arms give a dose-response — the blocked object’s appearance gap was +0.0272 and its cell moved 6 $\times$; the working object’s was +0.0089 and its cell did not move at all. A repair that improved every object would be consistent with generic regularization; one whose effect appears only where the measured defect is present, and vanishes where it is absent, is difficult to attribute to anything else. Note also what the null is made of: 21 of 50 trials flipped on a harness whose noise floor we measured at exactly zero, so the head genuinely changed behaviour while the outcome distribution did not. The repair is therefore *specific to objects whose deployment viewpoints have drifted*, not a general improvement to grasp prediction, and the working object’s remaining failures are something else we do not claim to have measured. **This scopes the word “grounding”**. The memorization audit and the vision-vs-proprioception attribution are untouched—they concern which *inputs* the grasp head uses and are measured independently—but the repaired head is grounded on *a box in the image*, not on *the named object*. “Object-level generalization” was consequently never the right frame for this campaign: the pipeline has no object-level channel to generalize over. A system that succeeds without discriminating its target is a weaker claim than one that succeeds by discriminating it, and we report the weaker true one.

Role-binding sub-study (four binders $\times$ three objects, $n=10$ each, one head and one configuration): the blocked object scores 0/10 under *every* binder—none, crop-CLIP rerank, mean prototype (offline identity 0.613), 1-NN bank (0.902)—while the two that cross move non-monotonically with offline accuracy (butter 4/7/2/4, soup 5/5/8/6). Full cells with run ids are in the manuscript appendix.

Offline binder accuracy dissociates from deployed success. The weaker binder owns the best soup cell and the worst butter cell; the binder with no offline number wins butter. A matched-episode A/B probe explains it: with the 0.902 binder active the uv stream feeding the grasp head *destabilises* (std 0.258 $\rightarrow$ 0.335 lateral, 0.047 $\rightarrow$ 0.202 vertical)—accurate on corpus crops, thrashing on live ones. The paper’s on-manifold limit again: a binder *built* from teacher trajectories cannot bind off-manifold crops.

Five predictions were pre-registered here and all five are recorded as falsified: that 0.82 per-tick identity accuracy would yield 0.914 median-of-3 latch correctness and hence cream > 0 under bank binding (the cell returned 0/10); that chocolate pudding would cross because it rarely won a misbind (its teacher never reached a grasp in five calibration iterations); the box-centre-spread prediction above; that candidate proposals would be too scarce for a binder to choose among (3.1–3.9 exist per frame); and that the instruction swap would be inert (it collapses the cell). Quantifying the deployed side of this boundary was attempted six times and abandoned six times, each at a pre-registered calibration gate: a projected-origin ground truth (rejected — soup, which succeeds 35/50, scored 0.111), its sign-corrected successor (rejected — the in-frame filter deletes the close-up target by construction), three text-tower discrimination probes (rejected — each failed to detect the working object’s own phrase), and the spread metric (rejected — confounded by camera motion, and inverted on the best-performing object). Deployed binding accuracy and text-tower discriminability are therefore reported as **unmeasured**, not estimated. The error is attributable—the prediction assumed offline per-tick accuracy equals deployed per-tick accuracy, which the probe refutes and which we do not measure. The prediction, its falsification, and this attribution are retained rather than removed.

6 Related work

6.1 Benchmark memorization and instruction-blindness

Both phenomena this paper reports have an established recent literature, and our contribution is smaller than a reader of our earlier draft would have concluded. LIBERO-PRO [25] perturbs objects, initial states, instructions and environments and finds that models exceeding 90% under standard evaluation collapse to **0.0%**, attributing it to “rote memorization of action sequences and environment layouts.” LIBERO-Plus [8] perturbs seven dimensions across ten VLAs and reports that models “tend to ignore language instructions completely.” LangGap [11] varies instruction semantics with the layout held fixed and reaches the same verdict for state-of-the-art models. Zhang and Bisk [24] name the failure *instruction blindness* and repair it optimizer-side; CAST [10] repairs it data-side with counterfactual labels, closely analogous to our command coverage.

Three consequences we state plainly. **(i)** That VLAs ignore language is *not* our finding; it is the consensus of this literature, and our system exhibiting it is confirmatory. **(ii)** The instruction-swap probe—change the instruction, hold the scene, rescore against the original task—is standard practice here and we withdraw it as a contribution. It remains in `eval/probes.py` because our *implementation* of it is annotation-free and its verdicts refuse over-claims, but the instrument is not ours. **(iii)** Our repair being data-side places it beside CAST rather than ahead of it.

What survives is narrower and, we think, still worth having. The cross-instruction *detection-agreement* probe (§5.2) asks whether two instructions’ prompt chains select the *same box from the same frame*. It needs no annotation, no counterfactual scene and no ground truth, and it interrogates the grounding *stage* rather than end-to-end behaviour — which is why it returned a usable answer where six ground-truth instruments died at their calibration gates. The mechanistic localization is likewise new to our knowledge: prior work establishes *that* language is ignored, while we show, for one system small enough to trace end to end, *where the language goes* — architecturally, in that `GraspPointHead.forward` takes no text argument at all, so the sole instruction $\rightarrow$ grasp path is box selection and that selection is identity-blind; and behaviourally, in that driving the detection prompts and the place-head embedding from *different* instructions isolates the collapse to one channel with no interaction (10/10 per-trial agreement). The language channel of a working policy reduces to a **single cached** (x, y) **coordinate**. Finally, the place-head layer and its repair are,

as far as we can tell, unreported: a benchmark in which every task shares one basket cannot expose a memorized command→location map, because memorized and grounded maps are behaviourally identical until the command changes.

How our perturbation regime relates to theirs. The two are doing different jobs and should not be read as competing magnitudes. LIBERO-PRO and LIBERO-Plus perturb *from outside*: many dimensions (objects, layouts, viewpoints, instructions, environments) across many policies, to establish that the scores collapse. Ours perturbs *one* dimension — the source object’s *xy* placement — on one policy, because the point is not to demonstrate collapse but to isolate a mechanism and then show a repair moving that mechanism. A multi-dimension sweep would reintroduce exactly the confound the single-dimension design removes.

The honest limitation is the magnitude, and it is ours rather than theirs. ± 4 cm was chosen to sit inside the shell’s ± 6 cm probe envelope — i.e. the perturbation was calibrated to the system under test, which caps what “passes randomization” can mean. We therefore no longer report a single radius: §5.7 sweeps $\pm 2/4/6/8$ cm on the same draws so the reader sees where each head degrades and where the envelope, rather than the head, becomes the binding constraint. Note also that a placement perturbation is the one axis on which LIBERO-Object has *no* headroom to be perturbed from inside: §1.2 shows its targets carry 2.11 bits of placement entropy against a 5.644-bit ceiling, so any placement variation at all must be introduced by the experimenter. On LIBERO-Spatial the same protocol would be redundant, since the suite already randomizes placement to ceiling.

One measurement does survive intact, and it is the one a referee reproduced rather than doubted. LIBERO-PRO characterizes the baseline qualitatively — variations “so subtle as to be visually imperceptible” — and reports no per-task position statistics, no distinct-value counts, no clustering and no precision analysis. Prior work *asserts* near-copying; Table 1 *measures* it, down to the ULP and including the orientation result that no prior work states.

6.2 Size

Size is comparable only under a stated convention, and the two in common use disagree about which system is smallest, because published counts routinely exclude a frozen language encoder that inference nonetheless loads. We report both (Table 9). Under *trained parameters* the smallest published alternative is Octo-small’s 27 M transformer [17], and MicroVLA’s 17.2 M is $1.6\times$ below it; under *deployed* = every parameter loaded at inference it is RT-1’s 35 M [4], and MicroVLA’s 30.2 M is below that *even after granting RT-1 a free language encoder*—RT-1’s count excludes the Universal Sentence Encoder producing its instruction embedding, for which we cite no figure rather than estimate one. MicroVLA is thus smallest under both conventions simultaneously, the strongest form of the claim the evidence supports. The survey was run adversarially, against systems whose stated goal is minimal size: NanoVLA-S is 52 M trainable atop a frozen ResNet18 *and* a frozen BERT-base (~ 161 M loaded) [6], and LiteVLA reaches a Raspberry Pi on a 256 M SmolVLM backbone [22]. Both sit above MicroVLA on both axes, and NanoVLA is the convention gap in miniature—a paper named for being nano, reporting 52 M while loading ~ 161 M, because the frozen language encoder is not counted.

Table 9: Size under both conventions. † excludes a frozen language encoder the published count omits. Sizes only: capability, task breadth, and protocol are not comparable.

system	trained	deployed	language encoder
RT-2 [5]	55 B	55 B	internal (PaLI-X)
OpenVLA [13]	7 B	7 B	internal (Llama-2)
SmolVLA [20]	450 M	450 M	internal (SmolVLM2)
LiteVLA [22]	LoRA only	~ 256 M	internal (SmolVLM-256M)
NanoVLA-S [6]	52 M	161 M	BERT-base, frozen
Octo-small [17]	<u>27 M</u>	138 M	T5-base, 111 M frozen
RT-1 [4]	35 M	<u>≥ 35 M</u> †	USE, frozen, uncounted
MicroVLA	17.2 M	30.2 M	<i>none</i> —detector’s own tower

The claim is bounded and checkable: it ranges over language-conditioned VLA policies with published parameter counts that we surveyed, it is a statement about parameter counts alone, and it is refuted by exhibiting any surveyed-class system below 17.2M trained or 30.2M deployed. We cannot rule out unpublished or unsurveyed smaller systems. “Language-conditioned” here names the *reference class* — systems that accept a language instruction — and is not a claim about how much work that instruction does in ours: the decomposition above shows it reduces to one latched place coordinate. A reader who thinks that disqualifies the artifact from the class should read the size row as smaller still, not larger. The mechanism behind the gap is architectural rather than incidental: MicroVLA carries no separate language model at all, because the detector’s own text tower supplies every text embedding—so the frozen encoder that the other rows pay for twice (once in memory, once in the convention mismatch) does not exist here. Of MicroVLA’s 17.2M trained parameters only 7.24M are causally load-bearing, since the 9.97M world model is inert in every cell we have ablated (§3). No number here is comparable to community-protocol LIBERO scores (600 steps, $n=10$, wrist-only, scripted-expert corpora). Predicting *where* rather than *how* is the Transporter [23] and CLIPort [19] lineage, continued by keypoint interfaces over engineered primitives [12, 15] and, with both levels learned, $\pi_{0.5}$ [2]. Our audit is a worked manipulation instance of causal confusion and copycat shortcuts [1, 7, 21]; what we add is the audited counterexample to probe-only certification, the resulting probe-plus-randomization pairing, and the control cells that scope the pairing itself. Bootstrapping from scripted experts is established [16]; our delta is that the scripted teacher is itself audited, caught encoding placement and detector version, and repaired.

7 Claims, non-claims, limitations

Claimed: the pinning measurement in its scoped form; the released head’s cells above; free per-tick regression at 0/56; the de-memorization mechanism at the head level (attribution shift, varied-label validation, and a paired jitter ablation, $0/10 \rightarrow 7/10$ dev, exact McNemar/sign $p \approx 0.016$; 9 discordant pairs, 8 in the repaired head’s favour, two-sided exact binomial), scoped by the audit-stack control; the seam defects of Table 4 and the taxonomy split (the count of 29 is our log tally, not a claim: 16 are named with anchors, 8 tabled here); the bounded size result of Table 9 (smallest of the surveyed systems under both the trained and the deployed convention simultaneously); and the identity-blindness of deployed role binding, together with its architectural corollary that no text embedding reaches the grasp head—claimed as a property *of this system*, and offered as a test other detector-grounded policies can run, not as a claim about them. **Not claimed:** any task-success contribution from the world model (causally inert in every cell we have ablated); end-to-end learned motor control (the shell is engineered); head-level grounding from the randomized column alone, stack-free; object-level generalization beyond the addendum’s two crossed objects (four attempted, three crossed; the fourth’s scripted teacher never reached a grasp, so no corpus exists for it and it is untested rather than failed); any deployed binding-accuracy figure beyond the world-space nearest-body counts; physical deployment; any size claim beyond Table 9’s bounded one (surveyed systems, published counts, parameters only)—neither that smaller systems cannot exist, nor that small size caused any result here. **Limitations:** $n=10$ per cell with $n=50$ confirmations on two protocols; ten held-out states reused across five disclosed selection looks; machine constants inherited from a dev-tuned era; substitution probes are on-manifold; ± 4 cm randomization sits inside the shell’s ± 6 cm probe envelope and separates heads only where visual goals are live; the teacher’s calibration consumed privileged telemetry offline; single benchmark, single simulator.

References

- [1] Mayank Bansal, Alex Krizhevsky, and Abhijit Ogale. Chauffeurnet: Learning to drive by imitating the best and synthesizing the worst. *arXiv preprint arXiv:1812.03079*, 2018.
- [2] Kevin Black et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [3] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for

- ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data*, 2017.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
 - [5] Anthony Brohan et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
 - [6] Jiahong Chen, Jing Wang, Long Chen, Chuwei Cai, and Jinghui Lu. NanoVLA: Routing decoupled vision-language understanding for nano-sized generalist robotic policies. *arXiv preprint arXiv:2510.25122*, 2025.
 - [7] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *NeurIPS*, 2019.
 - [8] Senyu Fei et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
 - [9] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.
 - [10] Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. Cast: Counterfactual labels improve instruction following in vision-language-action models. *arXiv preprint arXiv:2508.13446*, 2025.
 - [11] Y. Hou et al. Langgap: Diagnosing and closing the language gap in vision-language-action models. *arXiv preprint arXiv:2603.00592*, 2026.
 - [12] Wenlong Huang et al. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *CoRL*, 2023.
 - [13] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
 - [14] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *NeurIPS Datasets and Benchmarks*, 2023.
 - [15] Fangchen Liu, Kuan Fang, Pieter Abbeel, and Sergey Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *RSS*, 2024.
 - [16] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *CoRL*, 2023.
 - [17] Octo Model Team et al. Octo: An open-source generalist robot policy. In *RSS*, 2024.
 - [18] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *NeurIPS*, 2015.
 - [19] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. CLIPort: What and where pathways for robotic manipulation. In *CoRL*, 2021.
 - [20] Mustafa Shukor et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.

- [21] Chuan Wen, Jierui Lin, Trevor Darrell, Dinesh Jayaraman, and Yang Gao. Fighting copycat agents in behavioral cloning from observation histories. In *NeurIPS*, 2020.
- [22] Justin Williams, Kishor Datta Gupta, Roy George, and Mrinmoy Sarkar. Lite VLA: Efficient vision-language-action control on CPU-bound edge robots. *arXiv preprint arXiv:2511.05642*, 2025.
- [23] Andy Zeng et al. Transporter networks: Rearranging the visual world for robotic manipulation. In *CoRL*, 2020.
- [24] Haochen Zhang and Yonatan Bisk. Flatness preserves instruction following in vision-language-action models. *arXiv preprint arXiv:2606.23641*, 2026.
- [25] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.